

Round Robin vs Priority Comparison Project

- **Scenario A: Basic mixed workload**
- Four processes with varied burst times and priorities are scheduled using Round Robin with a time quantum of 3 ms.
-
-
- The CPU cycles through all processes equally, giving each a fixed slice before moving to the next.
-
-
- This baseline test shows how RR handles a typical mixed workload without any priority consideration.
-

- **Scenario A: Basic mixed workload**
- Four processes with varied burst times and priorities are scheduled using Round Robin with a time quantum of 3 ms.
-
-
- The CPU cycles through all processes equally, giving each a fixed slice before moving to the next.
-
-
- This baseline test shows how RR handles a typical mixed workload without any priority consideration.
-

Inputs:

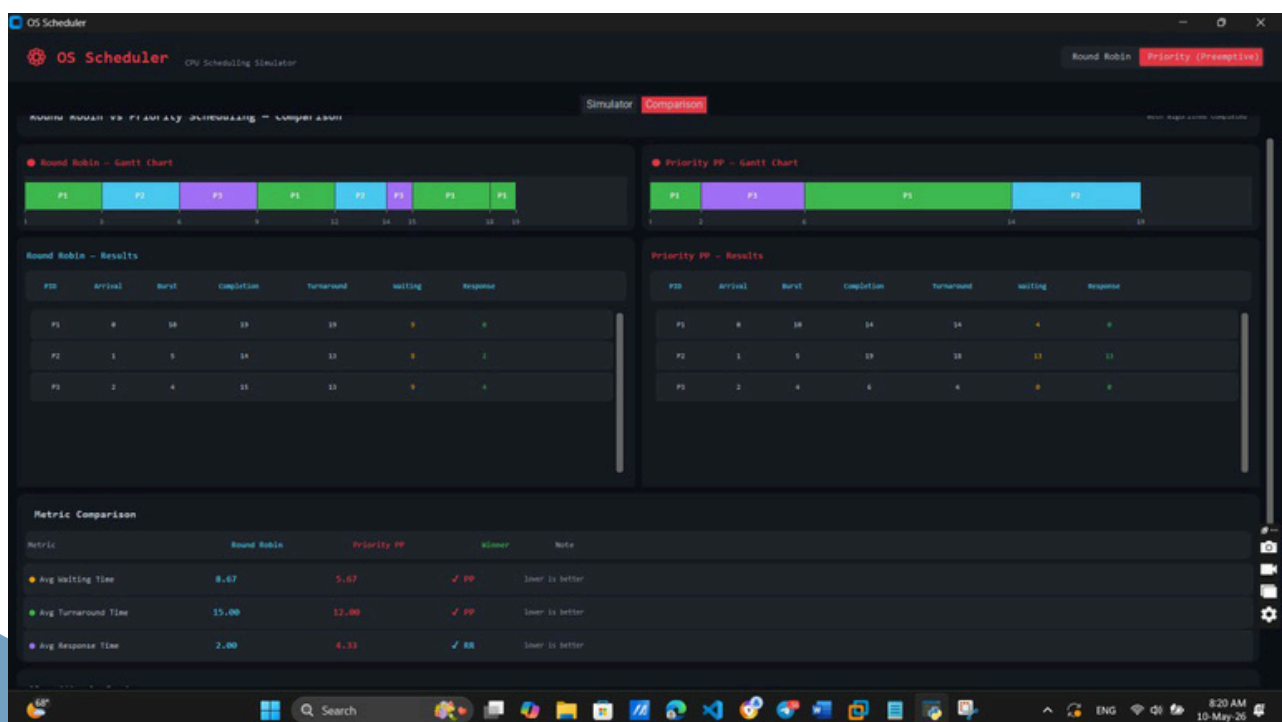
Process Table

✖ Clear All

+ Add

Process ID	Arrival Time	Burst Time	Priority	
P1	0	10	3	✖ 🔊
P2	1	5	3	✖ 🔊
P3	2	4	0	✖ 🔊

Round Robin (q=3) vs : priority



Analysis:

System Analysis

System Urgency:
Round robin is fair (equal CPU time sharing).
Priority is urgency-based (important tasks first).
Order:
Round robin changes execution dynamically based on priority.
Priority scheduling is cyclic order regardless of priority.
Process Benefit:
Priority scheduling benefits high priority processes more.
Risk:
Low priority processes may starve in Priority scheduling.
Round robin provides more balanced CPU distribution.

Scenario B: Urgency case

P2 and P4 carry priority 1 (highest), P1 has priority 2, and P3 has priority 3 (lowest)

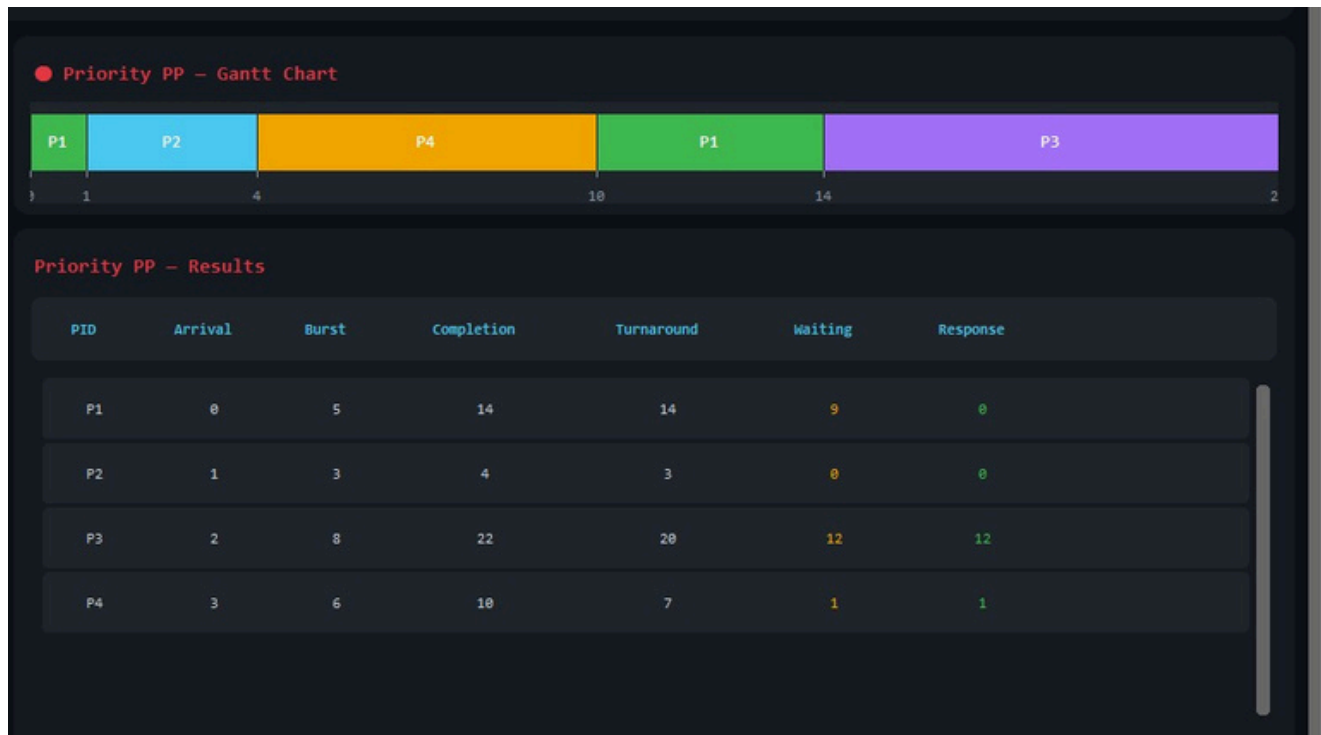
The Priority Preemptive scheduler immediately preempts lower-priority processes whenever a higher-priority process arrives

The urgency advantage is clearly visible in the waiting and response times of P2 and P4

Inputs:

Process Table					X Clear All	+ Add
Process ID	Arrival Time	Burst Time	Priority			
P1	0	5	2		X	
P2	1	3	1		X	
P3	2	8	3		X	
P4	3	6	1		X	

Result:

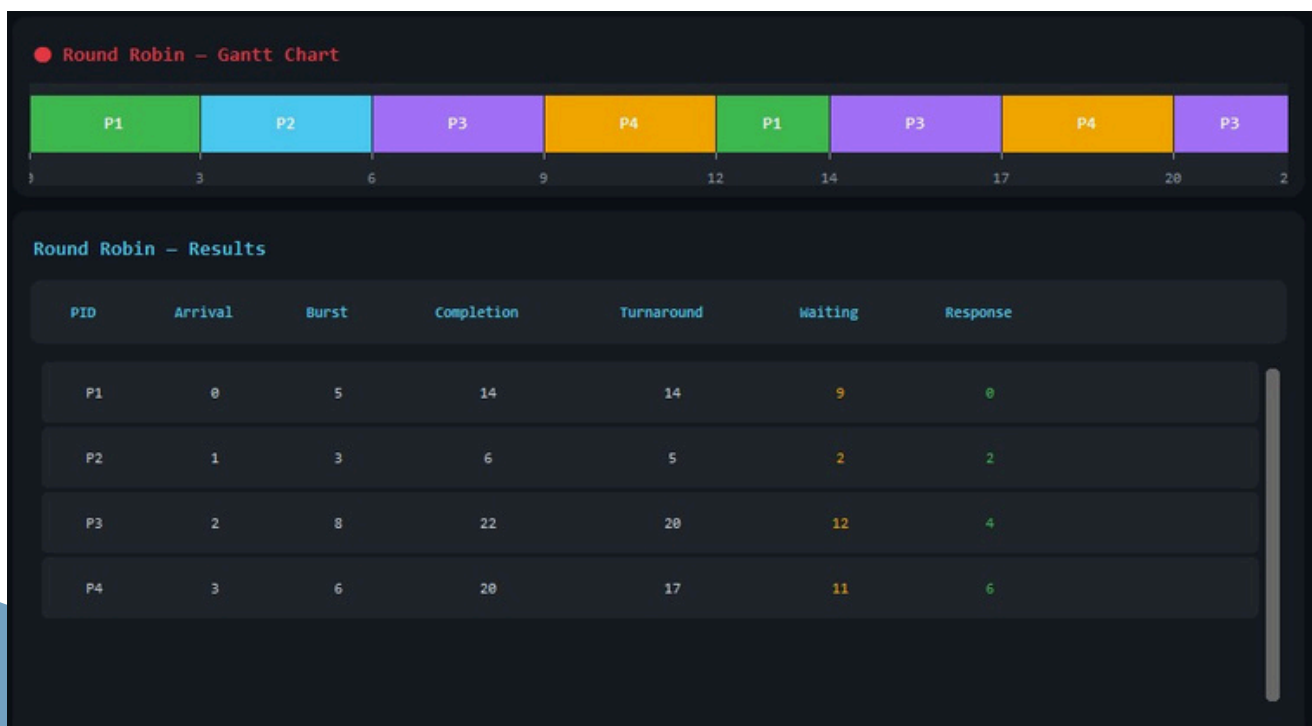


Conclusion

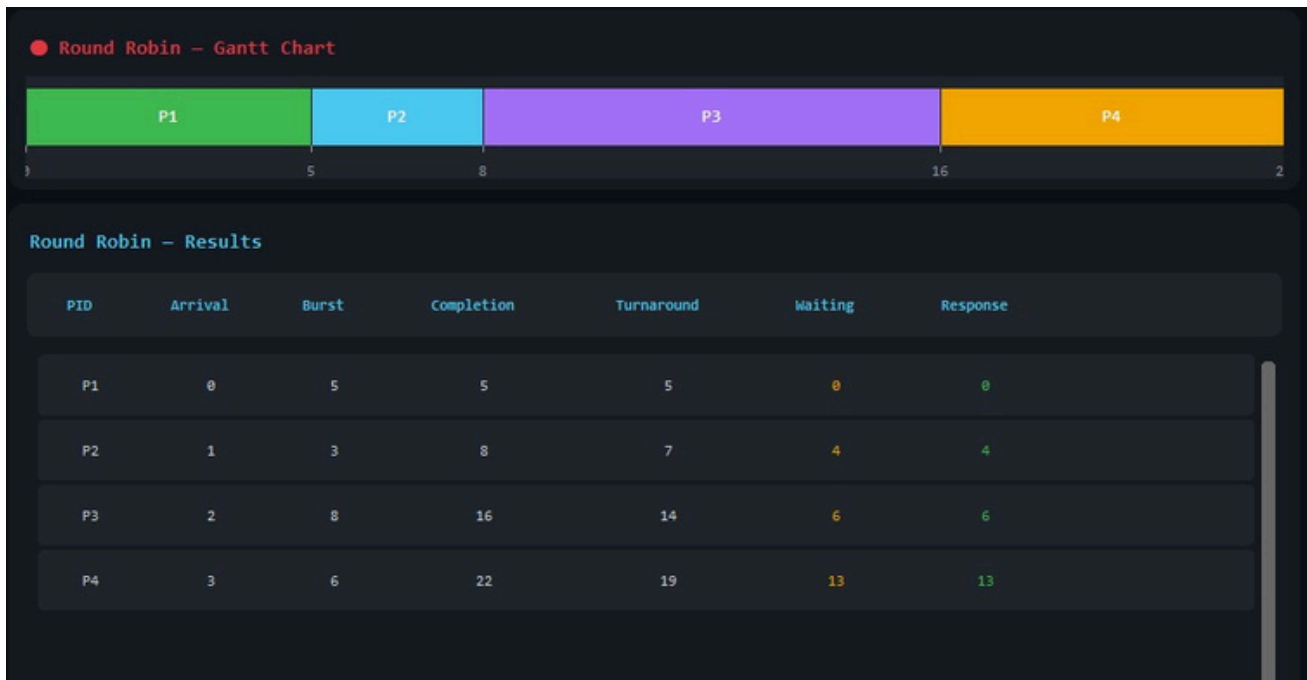
- P2 and P4 (priority 1) achieve waiting times of 0 and 1 ms respectively, confirming that high-priority processes receive near-immediate CPU service under Priority Scheduling

Scenario C: Fairness case

With $q=3$



With $q=10$



Conclusion:

- With a time quantum of 10 ms, every process fits within a single slice and runs to completion in arrival order
- This demonstrates Round Robin's fairness guarantee: no process is skipped due to priority
- Waiting times are bounded and predictable across all processes

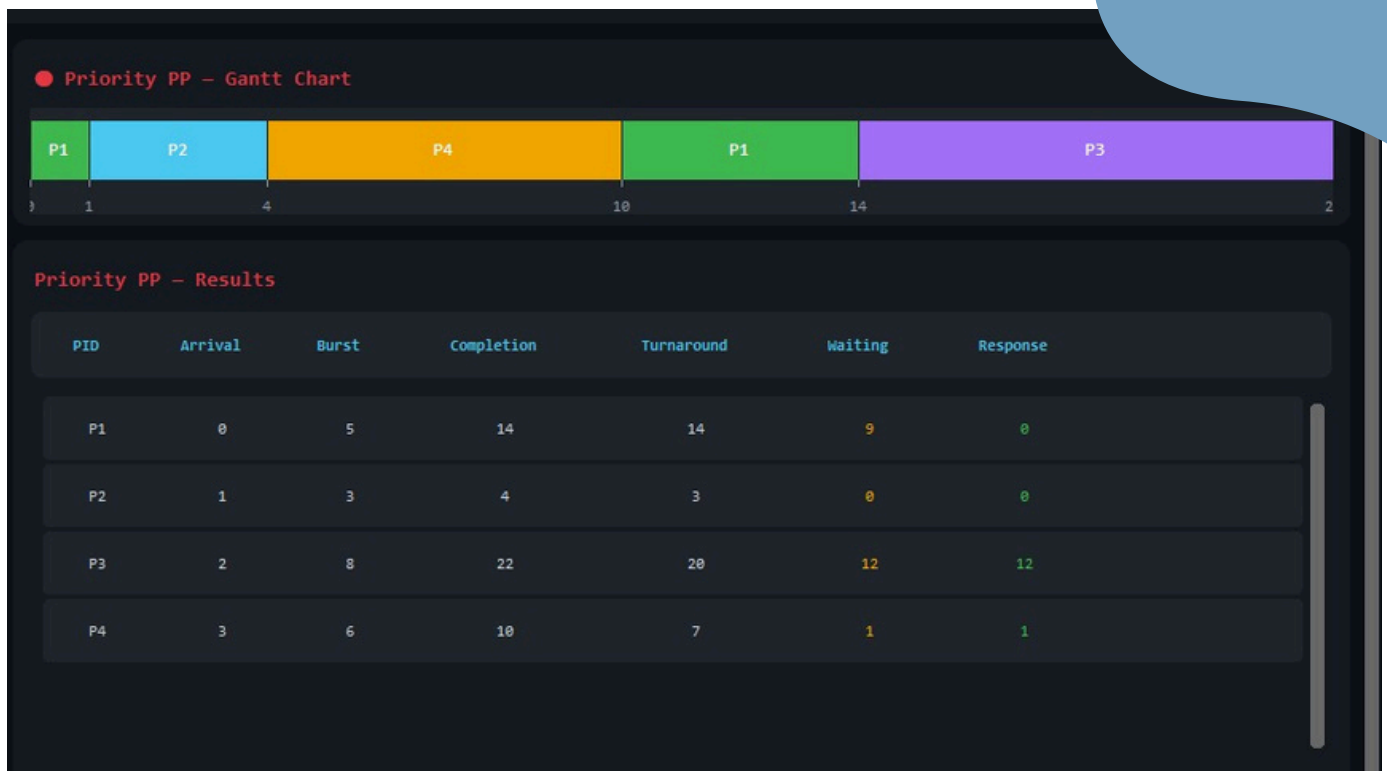
Scenario D: Possible starvation case

P3 carries the lowest priority (3) and the longest burst time (8 ms)

Because P2 and P4 (priority 1) and P1 (priority 2) are all served first, P3 is repeatedly pushed back

Its response time equals its waiting time - it never received a single CPU cycle until all higher-priority processes had finished

In a continuous workload with frequent high-priority arrivals, P3 would be indefinitely postponed, a clear starvation condition



Scenario E: Validation case

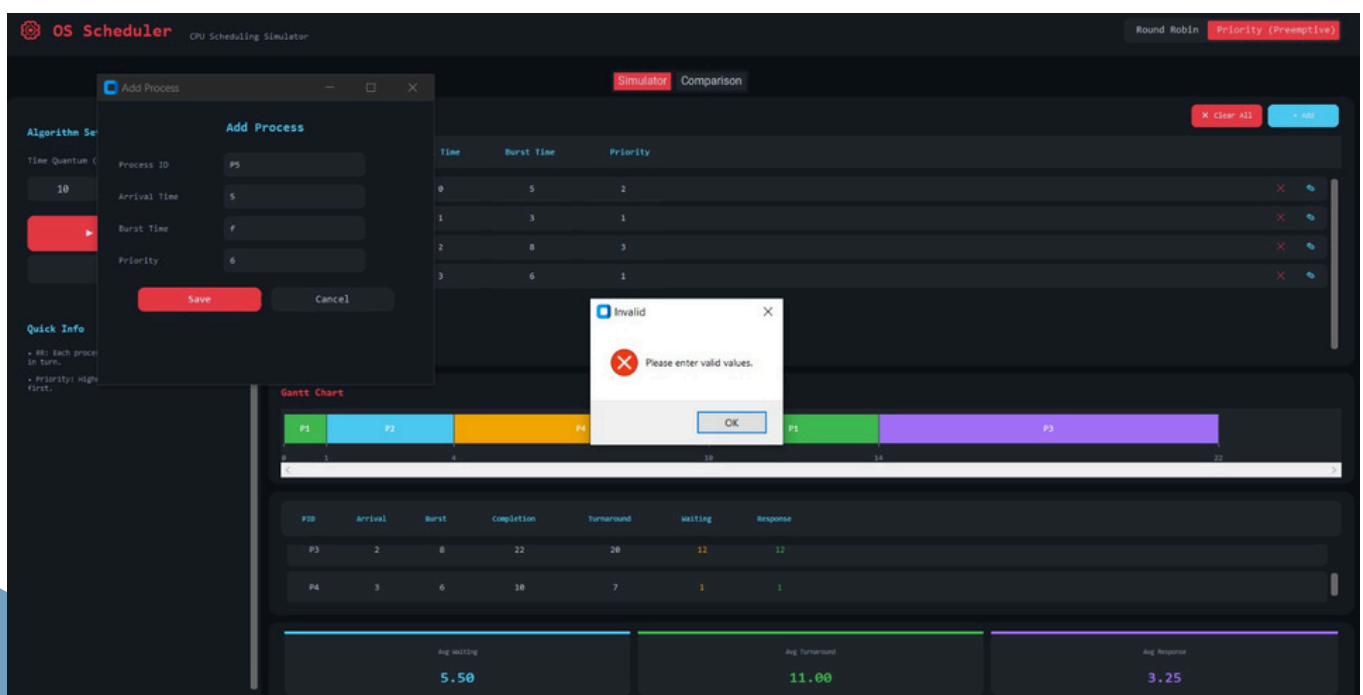
.Two invalid inputs were tested to verify the simulator's error handling

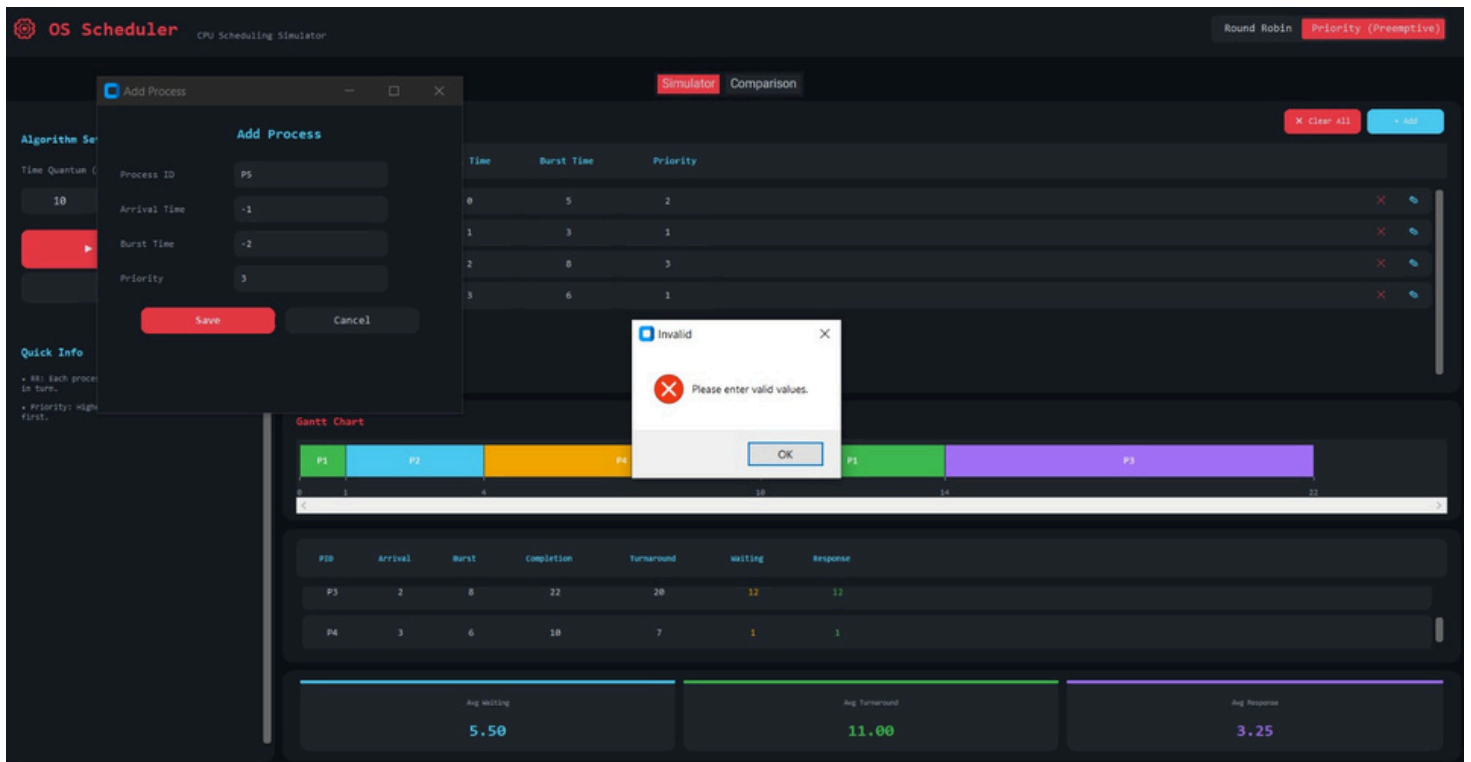
.In the first test, Burst Time was set to the letter 'f' (non-numeric)

.In the second test, Arrival Time and Burst Time were set to negative values (-1 and -2)

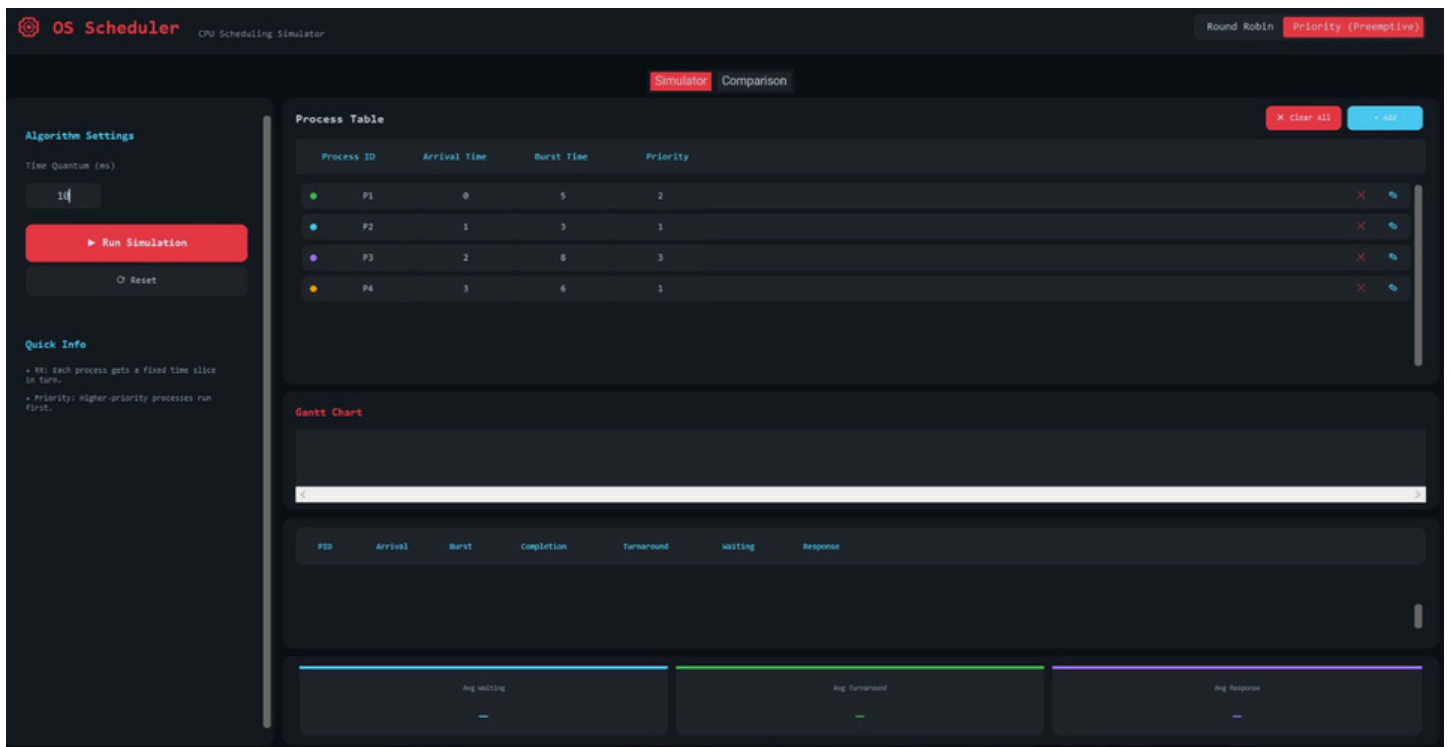
.".Both attempts were rejected immediately with the message: "Please enter valid values"

.No invalid process was added to the table





Reset case:



Required Analysis Questions

:

?Which algorithm gave better average time .1

Priority Preemptive gave a lower average waiting time of 5.50 ms compared to Round Robin's 8.50 ms and 5.75 ms (Q=10). Even at its best quantum setting, Round Robin could not match Priority (Q=3) Scheduling because preemption by high-priority processes reduces the overall queue time for the group

.as a whole

?Which algorithm gave better response time .2

Priority Preemptive achieved an average response time of 3.25 ms against Round Robin's 3.00 ms (Q=3) and 5.75 ms (Q=10). With a small quantum, Round Robin is marginally better for all processes on average; however Priority Scheduling delivers dramatically better response times for urgent processes

—

.P2 responded in 0 ms and P4 in 1 ms

?Did higher-priority processes gain significant advantage .3

Yes — clearly. Under Priority Preemptive, P2 (priority 1) waited 0 ms and responded in 0 ms; P4 (priority waited only 1 ms. Under Round Robin the same processes waited 2 ms and 11 ms respectively (1 because the scheduler ignores priority entirely. The advantage is both statistically significant and .operationally meaningful

Did Round Robin appear more balanced across all processes?Yes. Under Round Robin (Q=3) the .4 waiting times were 9, 2, 12, and 11 ms — a spread of 10 ms. Under

Priority Preemptive they were 9, 0, 12, and 1 ms — a spread of 12 ms with P3 bearing a disproportionate

share. Round Robin distributes CPU time more evenly because every process receives a guaranteed time

.slice regardless of priority

?Was starvation observed or likely in Priority Scheduling .5

Starvation was directly observed for P3. Its waiting time (12 ms) and response time (12 ms) were ,identical

meaning it never received CPU time until all higher-priority processes had finished. In a system with continuous high-priority arrivals, P3 would be indefinitely delayed — a clear starvation condition.

Round

.Robin eliminates this risk by design

?Which algorithm would you recommend for the tested workload, and why .6

For this workload, Priority Preemptive Scheduling is recommended. The workload contains time-sensitive

processes (P2, P4 at priority 1) that benefit greatly from immediate service. The algorithm delivers lower

average waiting time, significantly lower response time for urgent tasks, and better average .turnaround

The starvation risk for P3 can be mitigated by an aging mechanism without sacrificing urgency benefits. If

.all processes are equally important, Round Robin with a well-tuned quantum is the fairer choice

Required Conclusions

- **Performance winner Priority Preemptive Scheduling**
 - **outperformed Round Robin across all key metrics**
 - **Average waiting time 5.50 ms vs 5.75 ms ($Q = 10$)**
 - **Average turnaround time 11.00 ms vs 11.25 ms**
 - **Average response time 3.25 ms vs 5.75 ms**
- **Urgent-task handling Priority-based preemption**
 - **ensures that time-critical processes receive immediate CPU access**
- **Fairness Round Robin is more fair when all processes are considered equal as it ensures each process receives CPU time**
- **Starvation Starvation was observed for P3 (lowest priority) confirming that lower-priority processes may suffer indefinite delay under Priority Scheduling**